

رسالة محمد



یادگیری ماشین

مدرس: محمدرضا محمدی

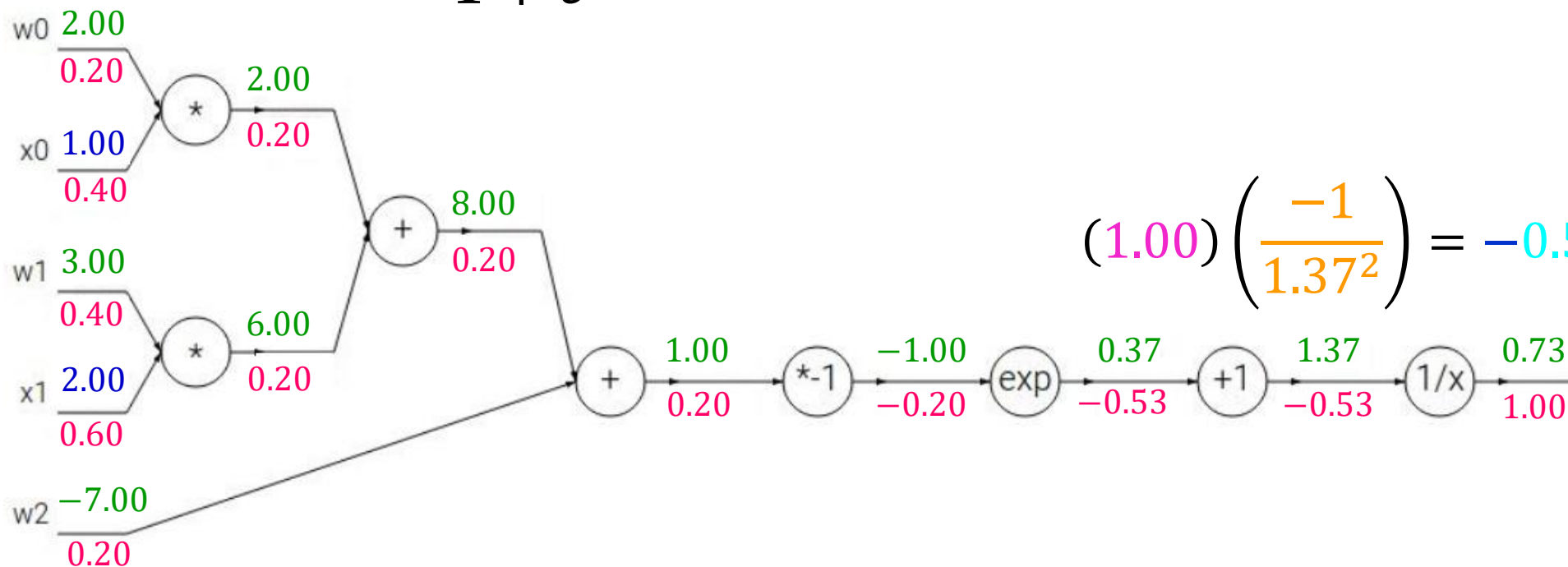
بهار ۱۴۰۲

شبکه‌های عصبی مصنوعی

Artificial Neural Networks

مثال: Linear + Sigmoid

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$\frac{\partial(ax)}{\partial x} = a$$

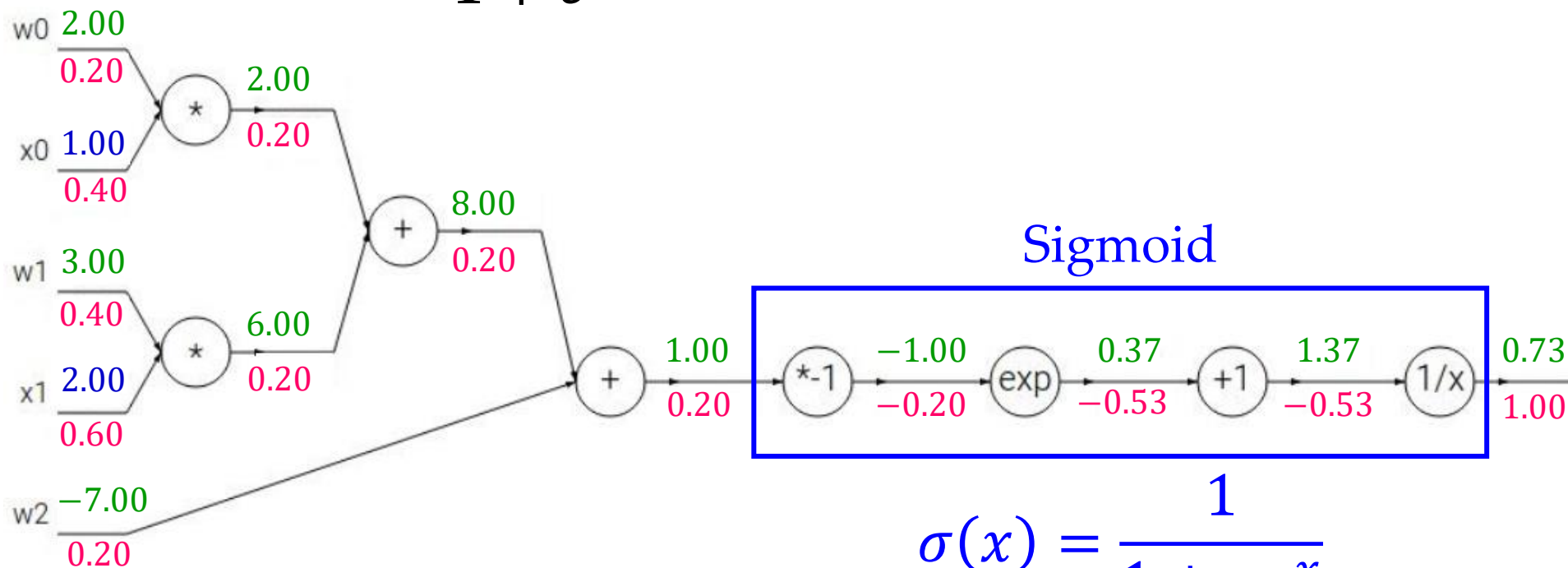
$$\frac{\partial(c + x)}{\partial x} = 1$$

$$\frac{\partial(e^x)}{\partial x} = e^x$$

$$\frac{\partial(1/x)}{\partial x} = \frac{-1}{x^2}$$

مثال: Linear + Sigmoid

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

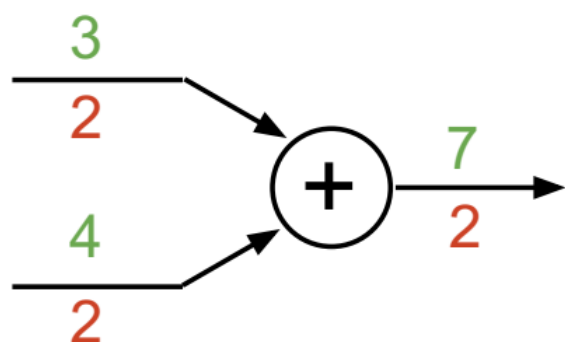


$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

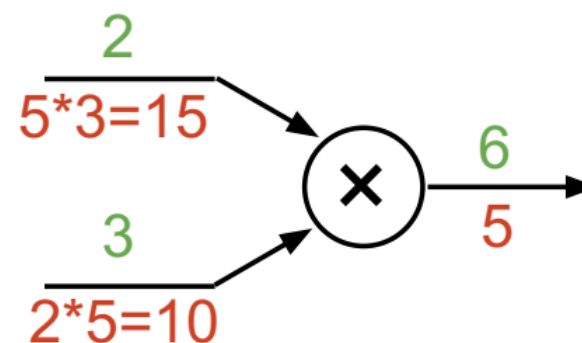
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1 - 1 + e^{-x}}{1 + e^{-x}} \frac{1}{1 + e^{-x}} = (1 - \sigma(x))\sigma(x)$$

نمونه‌هایی از جریان گرادیان

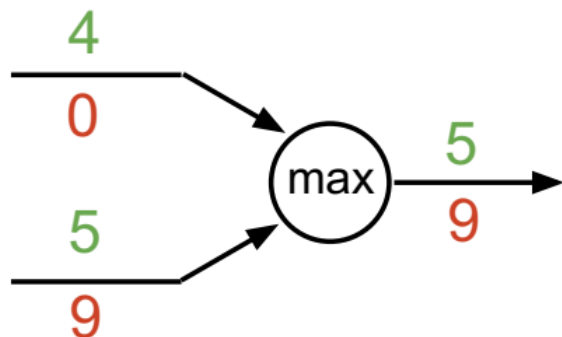
گیت جمع: توزیع کننده گرادیان



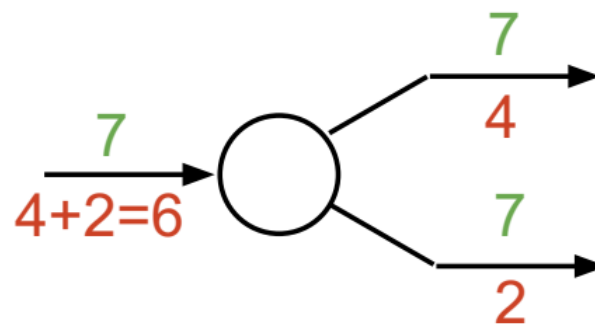
گیت ضرب: مبادله گر گرادیان



گیت بیشینه: روتر گرادیان



گیت کپی: جمع کننده گرادیان

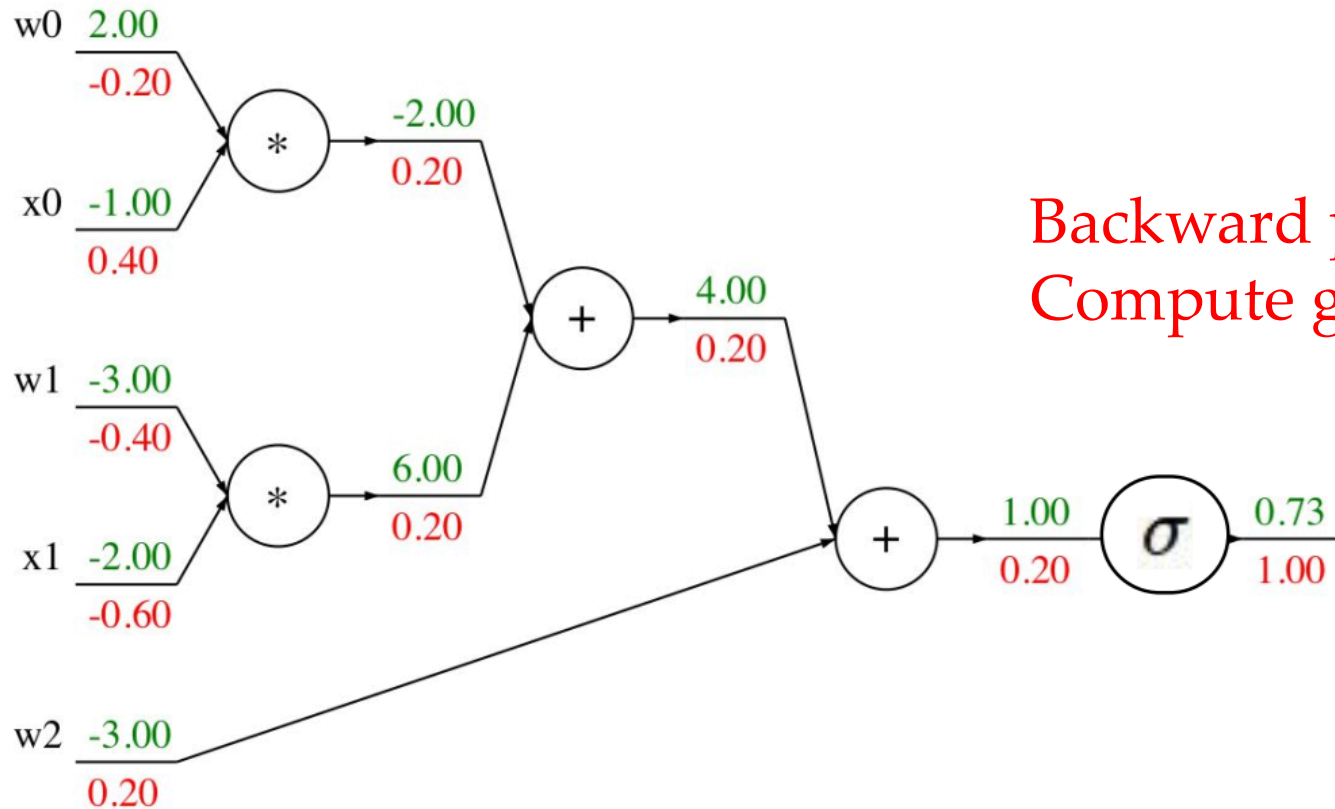


Forward pass:
Compute output

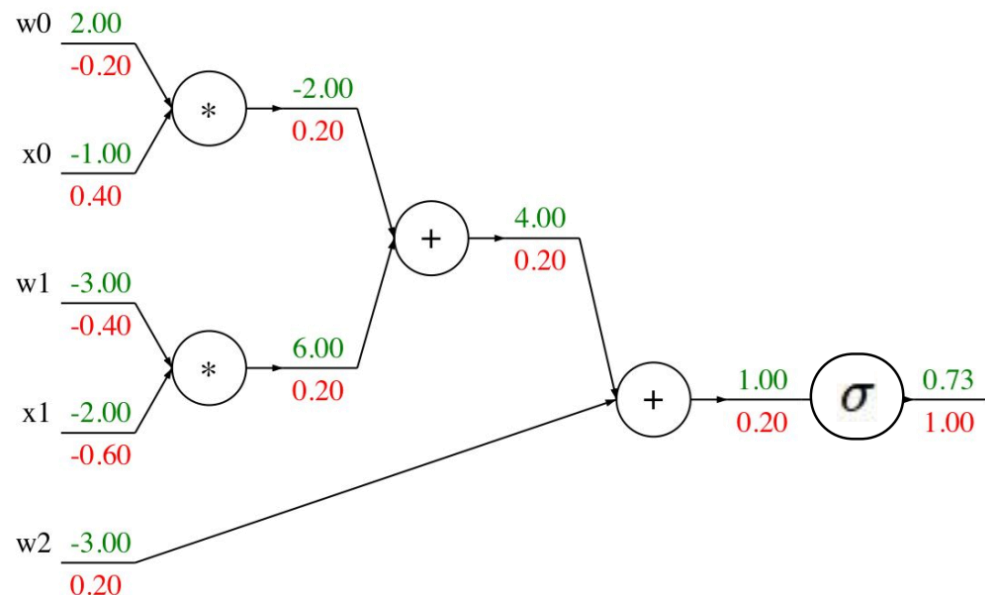
```
s0 = w0 * x0
s1 = w1 * x1
s2 = s0 + s1
s3 = s2 + w2
L = sigmoid(s3)
```

Backward pass:
Compute grads

```
grad_L = 1.0
grad_s3 = grad_L * (1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_w0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```



پیاده‌سازی ماژولار



```
class ComputationalGraph(object):  
    #...  
    def forward(inputs):  
        # 1. [pass inputs to input gates...]  
        # 2. forward the computational graph:  
        for gate in self.graph.nodes_topologically_sorted():  
            gate.forward()  
        return loss # the final gate in the graph outputs the loss  
    def backward():  
        for gate in reversed(self.graph.nodes_topologically_sorted()):  
            gate.backward() # little piece of backprop (chain rule applied)  
        return inputs_gradients
```


گیت ضرب

```
class MultiplyGate(object):
```

```
    """
```

```
    x,y are scalars
```

```
    """
```

```
    def forward(x,y):
```

```
        z = x*y
```

```
        self.x = x # Cache
```

```
        self.y = y # Cache
```

```
        # We cache x and y because we know that the derivatives contains them.
```

```
        return z
```

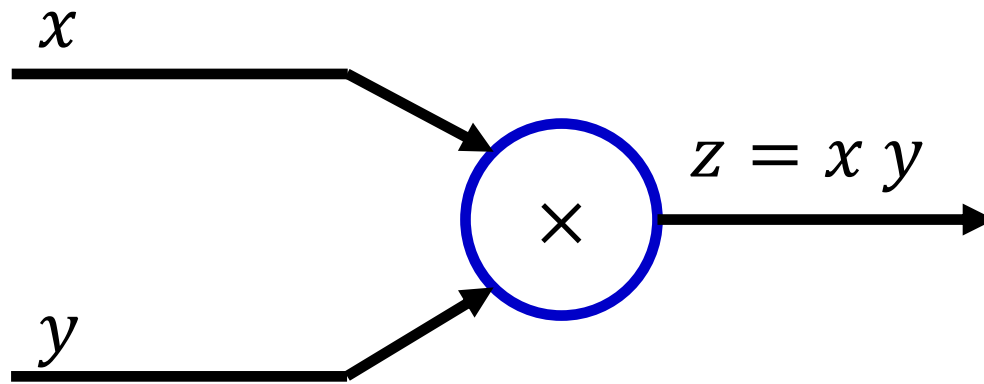
```
    def backward(dz):
```

```
        dx = self.y * dz
```

```
        #self.y is dx
```

```
        dy = self.x * dz
```

```
        return [dx, dy]
```



لایه‌ها در Caffe

BVLC / **caffe** Public

Notifications Star 32k Fork 18.9k

<> Code Issues 882 Pull requests 291 Actions Projects Wiki Security Insights

master caffe / src / caffe / layers / Go to file

 Noiredd Merge branch 'master' into patch_1	✓ 828dd10 on Aug 21, 2018	 History
..		
 absval_layer.cpp	dismantle layer headers	6 years ago
 absval_layer.cu	dismantle layer headers	6 years ago
 accuracy_layer.cpp	Added count==0 safeguard to CPU accuracy calculation	4 years ago
 accuracy_layer.cu	explain use of scratch diffs in comments	4 years ago
 argmax_layer.cpp	dismantle layer headers	6 years ago
 base_conv_layer.cpp	Shape mismatch CHECK logging improvements	5 years ago
 base_data_layer.cpp	Using default from proto for prefetch	5 years ago
 base_data_layer.cu	Switched multi-GPU to NCCL	5 years ago
 batch_norm_layer.cpp	CPU BatchNormLayer: replace powx with sqr and sqrt	5 years ago
 batch_norm_layer.cu	GPU BatchNormLayer: replace powx with mul and sqrt	5 years ago

لايه Sigmoid

```
template <typename Dtype>
void SigmoidLayer<Dtype>::Forward_cpu(const vector<Blob<Dtype>*>& bottom,
    const vector<Blob<Dtype>*>& top) {
    const Dtype* bottom_data = bottom[0]->cpu_data();
    Dtype* top_data = top[0]->mutable_cpu_data();
    const int count = bottom[0]->count();
    for (int i = 0; i < count; ++i) {
        top_data[i] = sigmoid(bottom_data[i]);
    }
}
```

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

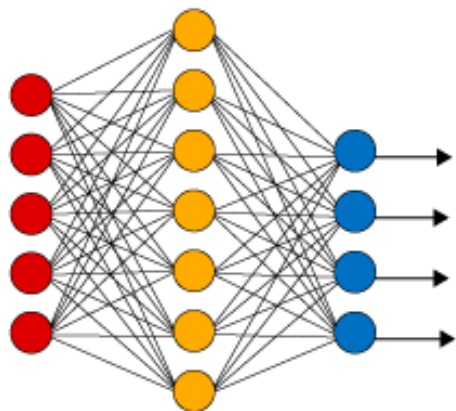
```
template <typename Dtype>
void SigmoidLayer<Dtype>::Backward_cpu(const vector<Blob<Dtype>*>& top,
    const vector<bool>& propagate_down,
    const vector<Blob<Dtype>*>& bottom) {
    if (propagate_down[0]) {
        const Dtype* top_data = top[0]->cpu_data();
        const Dtype* top_diff = top[0]->cpu_diff();
        Dtype* bottom_diff = bottom[0]->mutable_cpu_diff();
        const int count = bottom[0]->count();
        for (int i = 0; i < count; ++i) {
            const Dtype sigmoid_x = top_data[i];
            bottom_diff[i] = top_diff[i] * sigmoid_x * (1. - sigmoid_x);
        }
    }
}
```

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

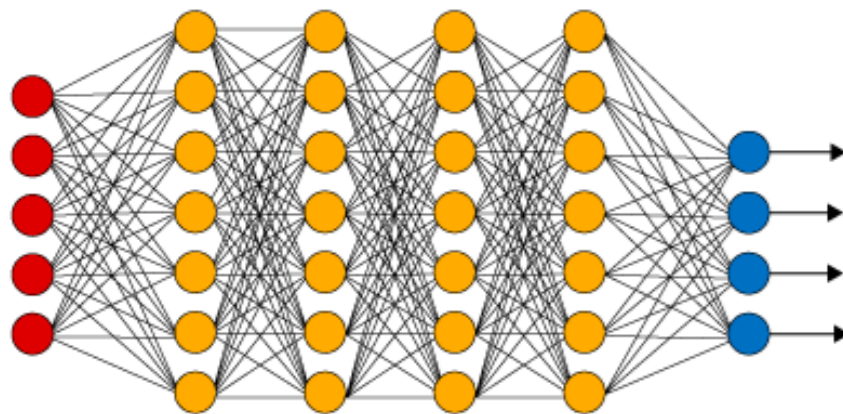
شبکه‌های عصبی عمیق

- آیا یک شبکه دارای لایه‌های زیاد می‌تواند منجر به بهبود طبقه‌بندی تصویر شود؟
- مهمترین ایراد این ساختار در پردازش تصویر آن است که اطلاعات همسایگی را لحاظ نمی‌کند
- به عبارت دیگر، دانش بدست آمده را میان پیکسل‌های تصویر به اشتراک نمی‌گذارد
- ایده اصلی در پیشرفت یادگیری عمیق در حوزه بینایی کامپیوتر استفاده از لایه‌های کانولوشنی است

Simple Neural Network



Deep Neural Network



● Input Layer

● Hidden Layer

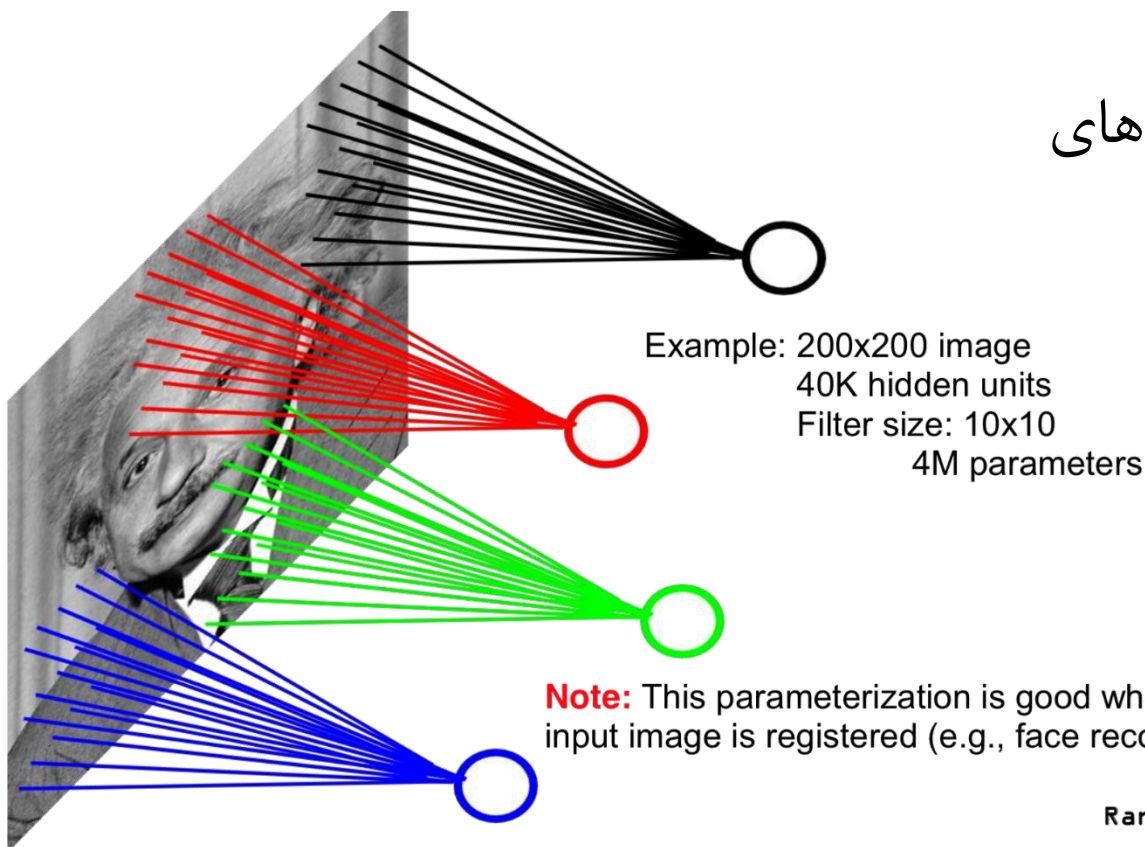
● Output Layer

شبکه‌های عصبی کانولوشنی

Convolutional Neural Networks

اتصال محلی

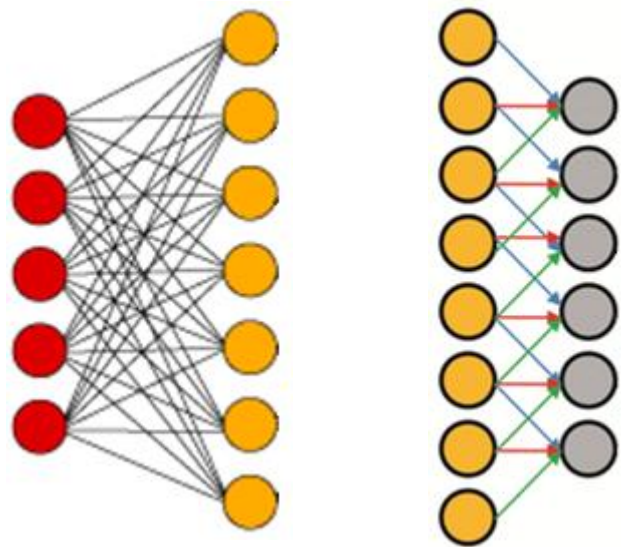
- برای استخراج ویژگی می توان هر نورون را به بخشی از تصویر متصل کرد
- تعداد پارامترها در این حالت به مراتب کاهش می یابد
- می توان وزن برخی نورون ها را مشترک قرار داد تا ویژگی های مشابهی را از بخش های مختلف تصویر استخراج کند



Note: This parameterization is good when input image is registered (e.g., face recognition).

کانولوشن

- در لایه‌های کاملاً متصل، مقدار هر نورون در لایه خروجی وابسته به تمام نورون‌ها در لایه قبل است
- کانولوشن یک‌بعدی مشابه با لایه کاملاً متصل است اما هر نورون خروجی تنها به بخشی از نورون‌های لایه ورودی متصل است



```
1 import numpy as np
2
3 a = [1, 2, 3, 4, 5]
4 b = [1, 2, 1]
5 np.convolve(a, b, 'valid')
```

```
array([ 8, 12, 16])
```

کانولوشن

- در پردازش تصاویر از کانولوشن دوبعدی استفاده می‌شود

3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

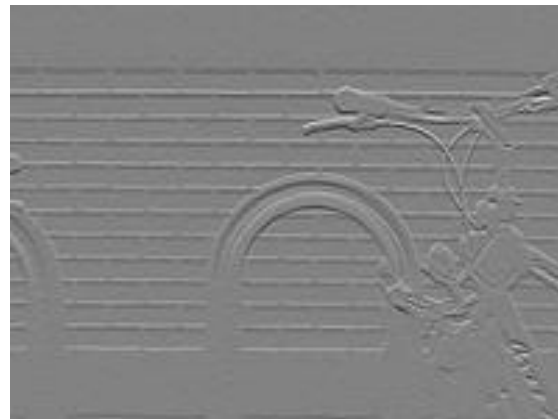
مثال: عملگر Sobel


$$G_y$$

+1	0	-1
+2	0	-2
+1	0	-1

$$G_x$$

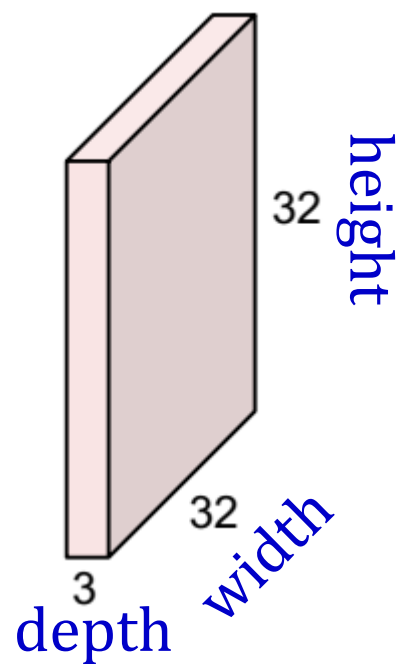
+1	+2	+1
0	0	0
-1	-2	-1



لایه کانولوشنی

خروجی لایه کانولوشنی حاصل فیلتر کردن
ماتریس ورودی با فیلتر مربوطه است که به
صورت مکانی بر روی آن لغزانده می شود

ورودی یک ماتریس
۳ بعدی است



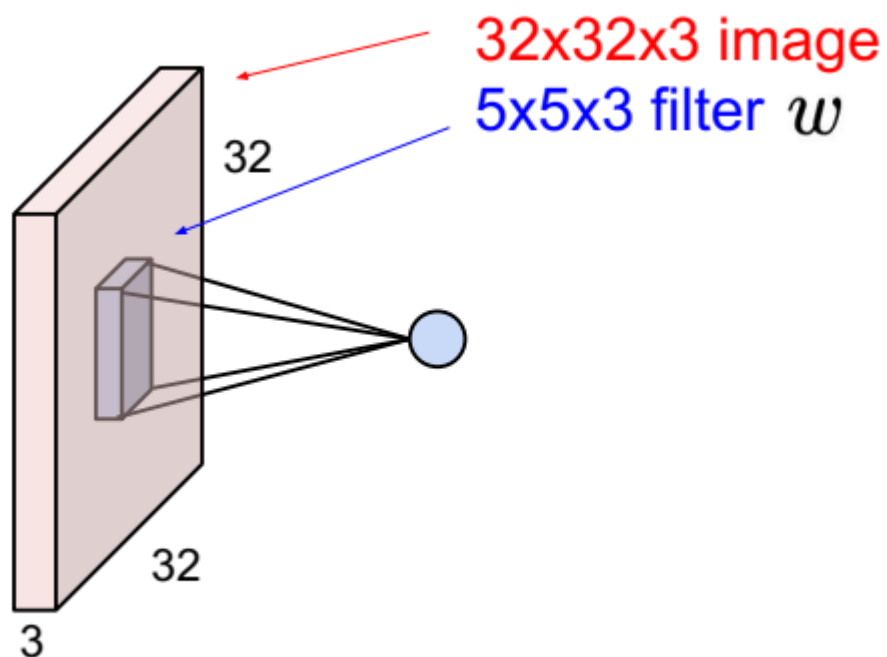
5x5x3 filter



لایه کانولوشنی

خروجی لایه کانولوشنی حاصل فیلتر کردن ماتریس ورودی با فیلتر مربوطه است که به صورت مکانی بر روی آن لغزانده می شود

ورودی یک ماتریس ۳ بعدی است



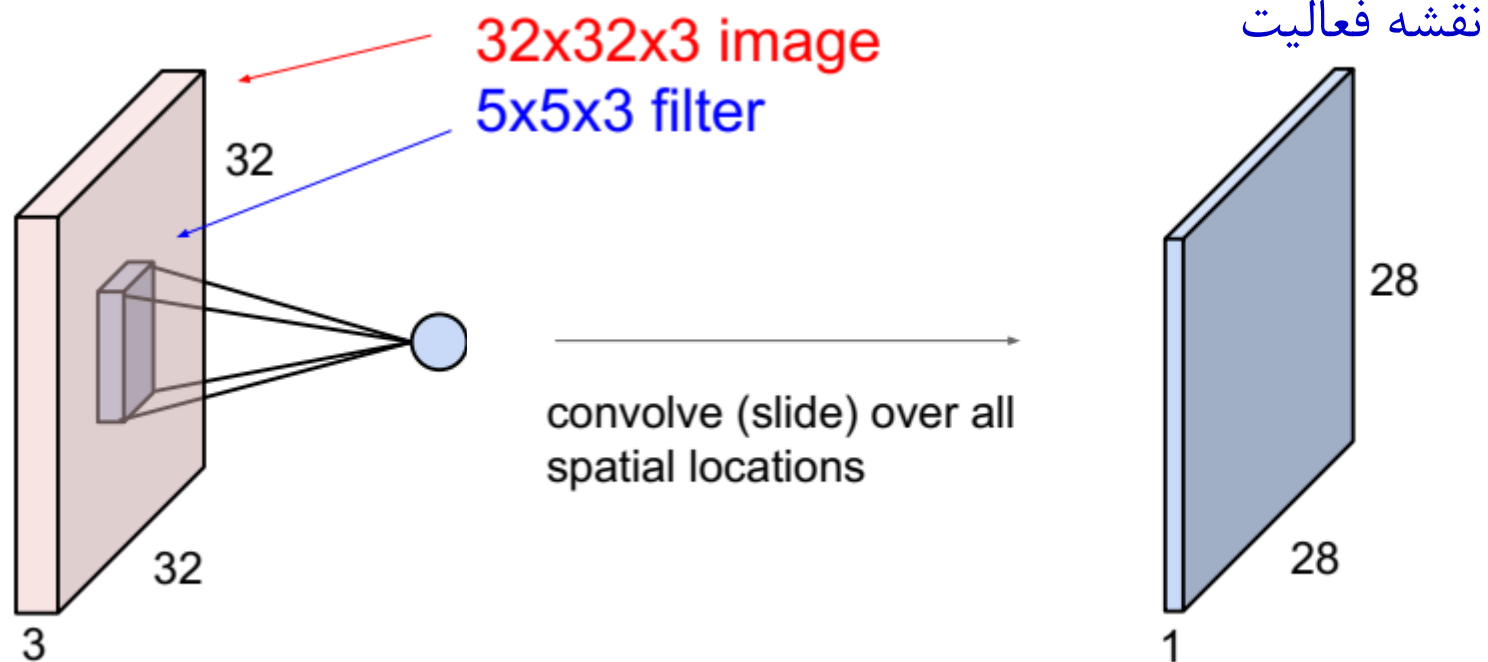
خروجی برابر با ضرب داخلی بین فیلتر و همسایگی مربوطه برای هر پیکسل است که معادل با ۷۵ ضرب و جمع است

$$w^T x + b$$

لایه کانولوشنی

خروجی لایه کانولوشنی حاصل فیلتر کردن
ماتریس ورودی با فیلتر مربوطه است که به
صورت مکانی بر روی آن لغزانده می شود

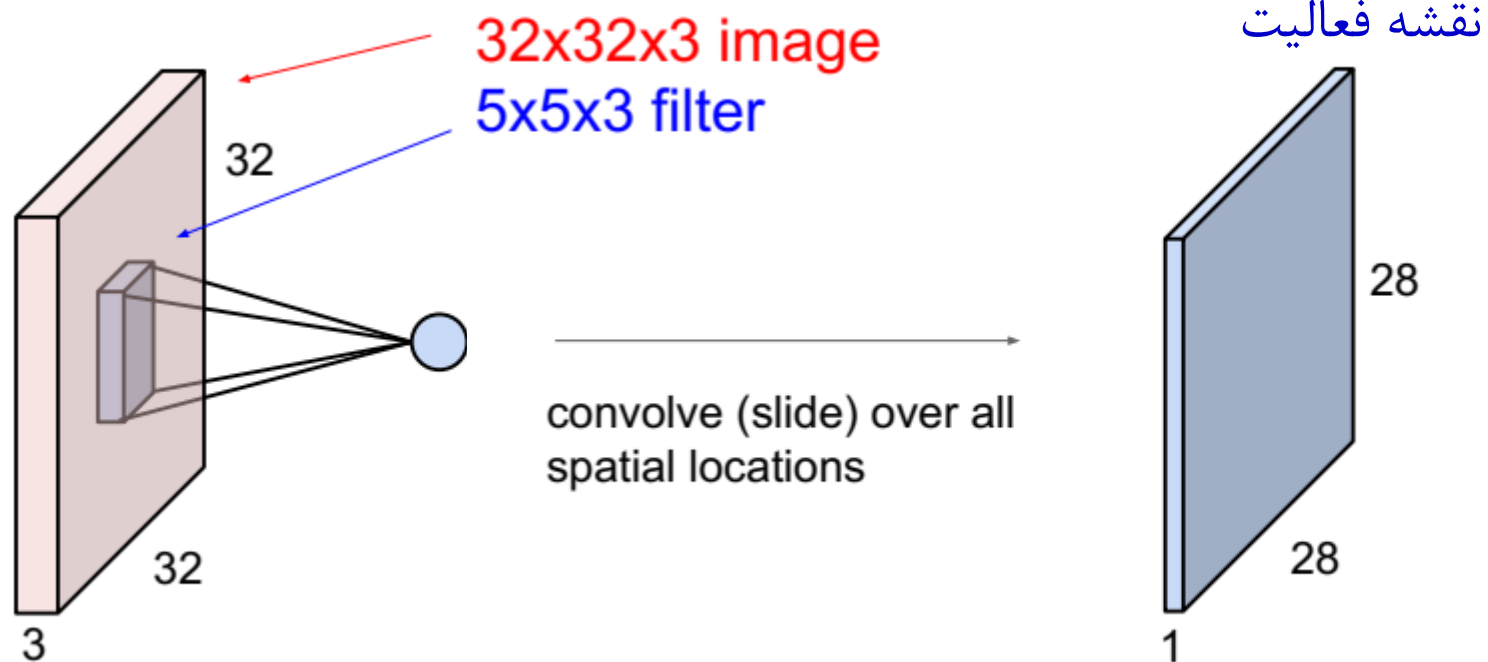
ورودی یک ماتریس
۳ بعدی است



لایه کانولوشنی

- البته یک فیلتر می‌تواند تنها یک مشخصه از تصویر را استخراج نماید

ورودی یک ماتریس
۳ بعدی است

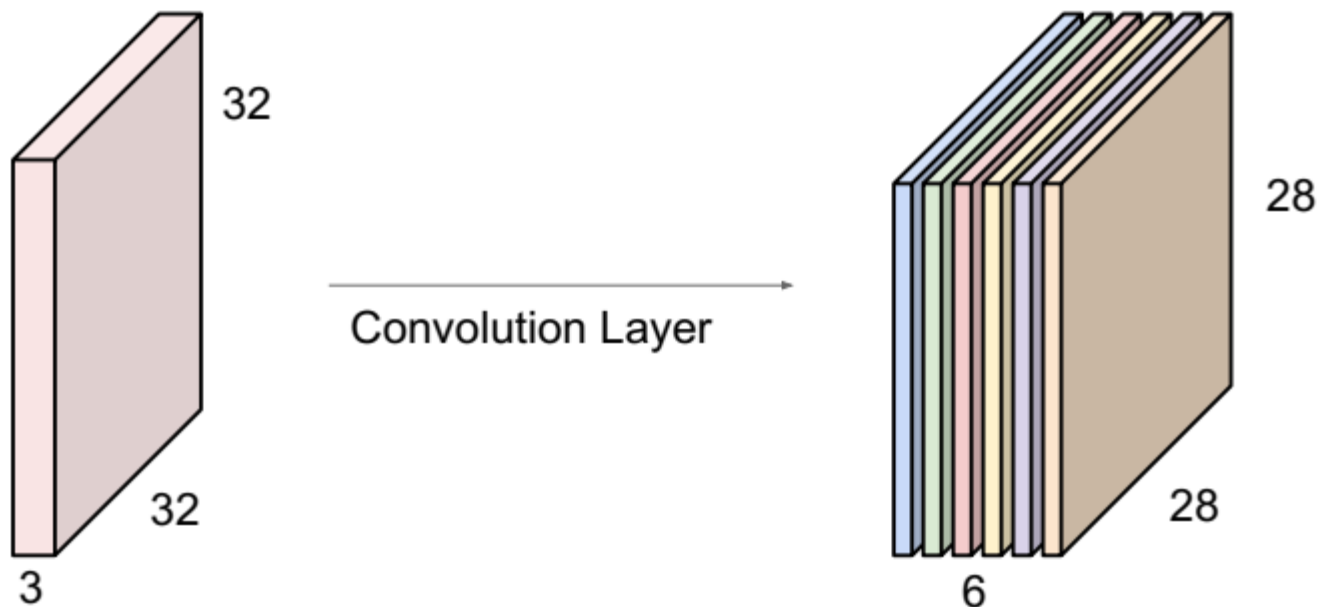


لایه کانولوشنی

- البته یک فیلتر می تواند تنها یک مشخصه از تصویر را استخراج نماید

ورودی یک ماتریس
۳ بعدی است

نقشه های فعالیت



لایه کانولوشنی در Keras

```
keras.layers.Conv2D(filters, kernel_size, strides=(1, 1), padding='valid',  
                    data_format=None, dilation_rate=(1, 1), activation=None,  
                    use_bias=True, kernel_initializer='glorot_uniform',  
                    bias_initializer='zeros', kernel_regularizer=None,  
                    bias_regularizer=None, activity_regularizer=None,  
                    kernel_constraint=None, bias_constraint=None)
```

filters: Integer, the dimensionality of the output space

kernel_size: Specifying the height and width of the 2D convolution window

activation: Activation function to use. If you don't specify anything, no activation is applied (see `keras.activations`)

https://colab.research.google.com/drive/10ygqxT8V2_-0fHHLWaeICka6hnlv1lao?usp=sharing

https://keras.io/api/layers/convolution_layers/convolution2d/